

Git distant avec GitHub (Collaboration et partage de projets)

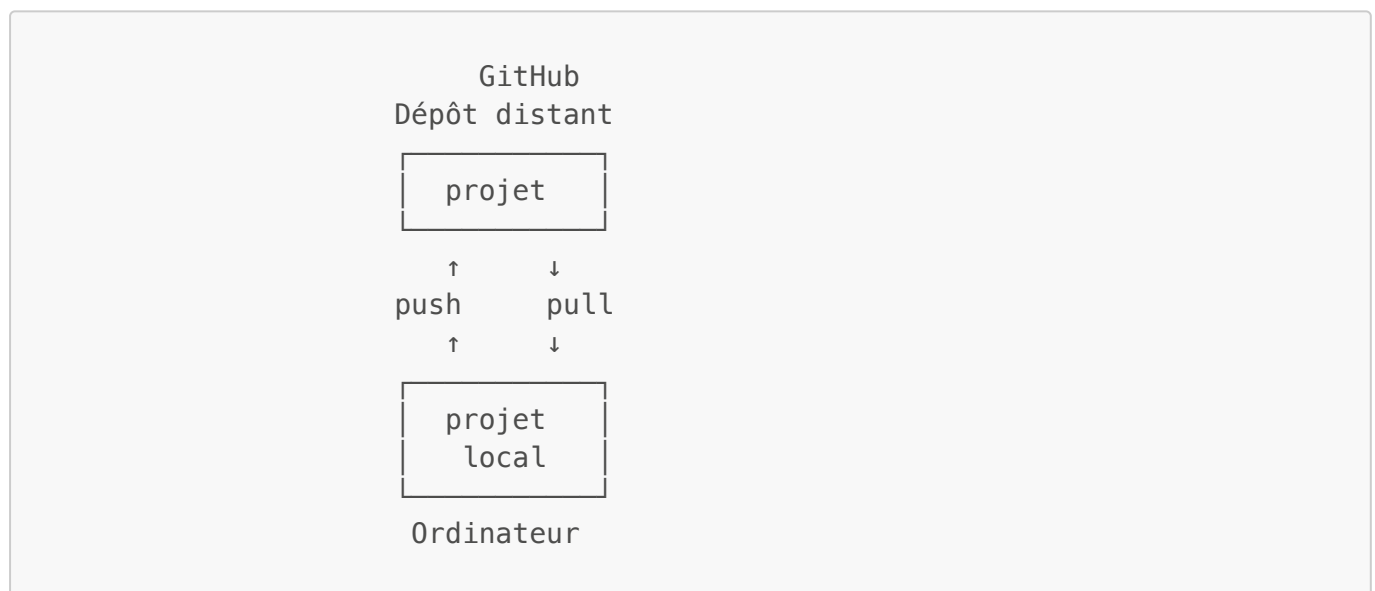
Git et GitHub :

- Git est un logiciel de gestion de versions. Il permet notamment de conserver l'historique des modifications d'un projet et de revenir à une version précédente si nécessaire.
- GitHub est une plateforme qui permet notamment d'héberger des dépôts Git sur Internet et de faciliter le travail collaboratif.

On peut donc avoir :

- un dépôt Git local, enregistré sur l'ordinateur ;
- un dépôt Git distant, hébergé par exemple sur GitHub.

Le dépôt local et le dépôt distant peuvent être synchronisés.



- Git : Gestionnaire de versions
- GitHub : Plateforme d'hébergement et de collaboration
- Dépôt local : Copie Git présente sur votre ordinateur
- Dépôt distant : Dépôt Git hébergé sur un serveur
- origin : Nom couramment utilisé pour désigner le dépôt distant principal

Créer un dépôt sur GitHub

1. Créer un compte GitHub : <https://github.com>, en utilisant votre adresse mail **carnus** ;
2. Connectez-vous à votre compte GitHub et créez un nouveau dépôt (bouton "New repository") ;

Pour les premiers exercices, nous allons créer un dépôt distant vide.

Lors de la création du dépôt :

- choisissez le nom du TP : **tp1_git** ;

- choisissez Public ;
- ne cochez pas Add a README file ;
- ne cochez pas Add .gitignore ;
- ne choisissez pas de licence pour le moment.

Après la création du dépôt, GitHub affiche son adresse, par exemple :

https://github.com/votre-pseudo/tp1_git.git

Cette adresse sera utilisée pour associer le dépôt local au dépôt distant.

Associer un dépôt local à GitHub (dépôt distant)

Placez-vous dans le dossier du projet :

```
cd tp1_git
```

Vérifier l'état du dépôt :

```
git status
```

Pour associer le dépôt local au dépôt GitHub, utilisez :

```
git remote add origin https://github.com/votre-pseudo/tp1_git.git
```

- **remote** concerne les dépôts distants ;
- **add** signifie que l'on ajoute un dépôt distant ;
- **origin** est le nom donné au dépôt distant ;
- l'URL correspond à l'adresse du dépôt GitHub.
- Vérifier le dépôt distant

```
git remote -v
```

Vous devriez obtenir quelque chose de similaire à :

```
origin  https://github.com/votre-pseudo/tp1_git.git (fetch)
origin  https://github.com/votre-pseudo/tp1_git.git (push)
```

Le dépôt local est maintenant associé au dépôt GitHub.

HTTPS ou SSH ?

Il existe plusieurs moyens de communiquer avec GitHub.

Avec HTTPS

Exemple :

```
git remote add origin https://github.com/votre-pseudo/tp1_git.git
```

Avec SSH

Exemple :

```
git remote add origin git@github.com:votre-pseudo/tp1_git.git
```

SSH permet de s'authentifier à l'aide de clés cryptographiques.

Cette méthode nécessite une configuration préalable des clés SSH.

Envoyer son projet vers GitHub : `git push`

Une fois le dépôt local associé au dépôt distant, nous pouvons envoyer nos commits vers GitHub.

Notre branche principale s'appelle `main`.

```
git push -u origin main
```

- `git push` : envoyer des commits
- `origin` : vers le dépôt distant appelé origin
- `main` : depuis la branche main
- `-u` : mémoriser la branche distante associée

Le `-u` signifie `--set-upstream`.

Il permet d'associer la branche locale main à la branche distante correspondante.

Lors des prochains envois, la commande suivante suffira généralement :

```
git push
```

Vérifier le résultat sur GitHub

Actualisez la page de votre dépôt GitHub.

Vous devriez maintenant voir les fichiers du TP.

Le dépôt GitHub contient une copie de l'historique Git du TP.

Vous pouvez notamment consulter :

- les fichiers ;
- les commits ;
- les branches ;
- les modifications apportées au TP.

Récupérer les modifications : `git pull`

Modifier le fichier HTML GitHub en ajoutant le lien pour utiliser les icônes `fontawesome`.

Pour récupérer les modifications :

```
git pull
```

`git pull` permet de récupérer les modifications du dépôt distant et de les intégrer à votre branche locale.



Cependant, `git pull` peut provoquer un conflit si vos modifications locales et les modifications distantes concernent les mêmes lignes.

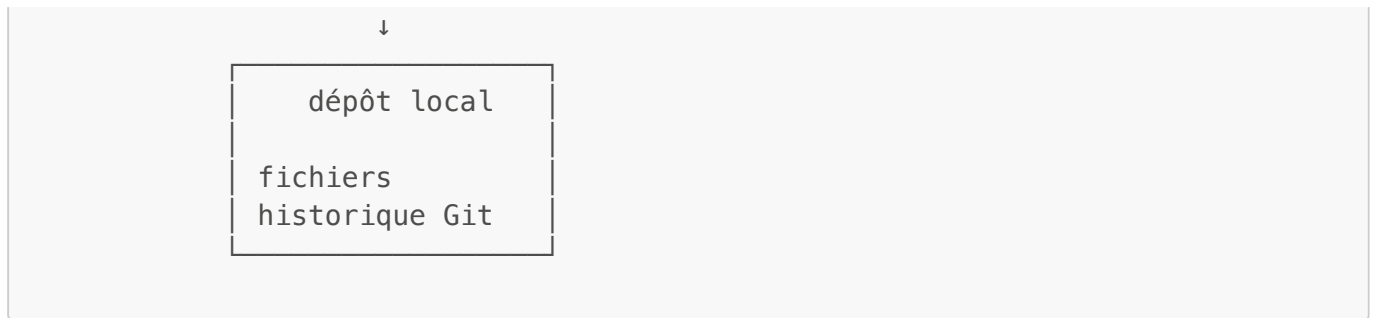
Cloner un projet GitHub

Depuis GitHub, pour **télécharger un projet distant** :

```
git clone https://github.com/AutreUtilisateur/le-projet.git
```

Cela crée automatiquement un dossier avec un dépôt Git lié au GitHub distant.

```
GitHub
dépôt distant
|
| git clone
```



`git clone` est généralement utilisé pour obtenir une première copie locale d'un dépôt distant.

`git pull` est ensuite utilisé pour récupérer les nouvelles modifications.

Les commandes essentielles

Commande	Fonction
<code>git status</code>	Afficher l'état du dépôt
<code>git add</code>	Ajouter des modifications à la zone de staging
<code>git commit</code>	Enregistrer les modifications dans l'historique
<code>git log</code>	Consulter l'historique
<code>git remote -v</code>	Afficher les dépôts distants
<code>git push</code>	Envoyer des commits vers le dépôt distant
<code>git pull</code>	Récupérer les modifications du dépôt distant
<code>git clone</code>	Cloner un dépôt distant
<code>git switch</code>	Changer de branche
<code>git merge</code>	Fusionner une branche

TP 1 : Créer et publier un projet personnel sur GitHub**

Objectif :

Créer un petit site HTML/CSS, suivre son évolution avec Git et publier le projet sur GitHub.

Étape 1 — Créer le projet

1. Créer un dossier appelé `MonSiteWeb`.
2. Dans ce dossier, ajouter un fichier `index.html` contenant un squelette HTML basique.

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
<title>Mon site Web</title>
</head>
<body>
  <h1>Bienvenue sur mon site</h1>
  <p>Je suis étudiant en BTS CIEL – IR.</p>
</body>
</html>
```

Étape 2 — Initialiser le dépôt Git

1. Ouvrir Git Bash (ou le terminal de VSCode) dans le dossier **MonSiteWeb**.

```
git init
git status
```

Git doit signaler que index.html n'est pas encore suivi.

Étape 3 — Ajouter le fichier à la zone de staging

4. Créer un dépôt sur GitHub (nom : **MonSiteWeb**).

```
git add index.html
git status
```

Le fichier doit maintenant apparaître comme prêt à être commité.

La zone de staging, également appelée index, permet de sélectionner les modifications qui seront incluses dans le prochain commit.

Étape 4 — Créer le premier commit

```
git commit -m "Ajout de la page d'accueil"
```

Vérifier l'historique :

```
git log --oneline
```

Étape 5 — Créer le dépôt GitHub

Sur GitHub, créer un dépôt appelé : **MonSiteWeb**

Ne pas créer de README ou de .gitignore depuis GitHub.

Étape 6 — Associer les deux dépôts

Utiliser l'URL correspondant à votre dépôt :

```
git remote add origin https://github.com/votre-pseudo/MonSiteWeb.git
```

Vérifier :

```
git remote -v
```

Étape 7 — Publier le projet

Envoyer la branche main vers GitHub :

```
git push -u origin main
```

Vérifier ensuite le résultat directement sur GitHub.

Étape 8 — Modifier le projet

Ajouter un fichier : `style.css`

```
body {  
  font-family: Arial, sans-serif;  
  background-color: #f4f4f4;  
  color: #333;  
  text-align: center;  
}  
  
h1 {  
  color: #007bff;  
}
```

Modifier également `index.html` pour charger la feuille de style :

Étape 9 — Enregistrer les modifications

Commencer par observer l'état du dépôt :

```
git status
```

Ajouter les modifications :

```
git add .
```

Créer un commit :

```
git commit -m "Ajout de la feuille de style"
```

Envoyer vers GitHub :

```
git push
```

Vérifier le résultat sur GitHub.

Étape 10 - Utiliser un fichier `.gitignore`

Créer un fichier nommé : `.gitignore`

Ajouter :

```
*.log  
node_modules/
```

Cela signifie que Git ignorera :

- les fichiers dont l'extension est `.log` ;
- le dossier `node_modules`.

Pourquoi peut-il être intéressant de ne pas versionner certains fichiers ou dossiers ?

TP 2 : Travailler avec les branches et résoudre un conflit

Objectif :

Découvrir les branches et comprendre comment plusieurs versions d'un projet peuvent évoluer indépendamment.

Une branche permet de travailler sur une évolution du projet sans modifier directement la branche principale.

Nous allons utiliser `main` comme branche principale.

Étape 1 — Vérifier la branche actuelle


```
git branch
```

La branche courante est indiquée par *****.

Étape 2 — Créer une branche

Créer une branche appelée **modif-texte** avec :

```
git switch -c modif-texte
```

Vérifier :

```
git branch
```

Vous devez obtenir quelque chose de similaire :

```
* modif-texte  
main
```

Vous êtes maintenant sur la branche **modif-texte**.

Étape 3 — Modifier le projet

Dans **index.html**, modifier le titre :

```
<title>Bienvenue sur mon site</title>
```

Enregistrer le fichier.

Vérifier :

```
git status
```

Ajouter la modification :

```
git add index.html
```

Créer le commit :

```
git commit -m "Modification du titre"
```

Étape 4 — Revenir sur main

Retourner sur la branche principale :

```
git switch main
```

Vérifier :

```
git branch
```

La branche active doit maintenant être :

```
* main  
  modif-texte
```

Étape 5 — Effectuer une autre modification

Sur la branche `main`, modifier le même titre, mais avec une autre valeur :

```
<title>Mon site officiel</title>
```

Enregistrer puis créer un commit :

```
git add index.html  
git commit -m "Modification du titre sur main"
```

Étape 6 — Fusionner deux branches

Nous avons maintenant deux branches qui ont évolué différemment.



Nous sommes actuellement sur main.

Demandons à Git de fusionner la branche `modif-texte` :

```
git merge modif-texte
```

Comme les deux branches ont modifié la même ligne de manière différente, Git doit normalement signaler un conflit.

Un message similaire à celui-ci peut apparaître :

```
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts and then commit the result.
```

Comprendre un conflit Git

Git peut afficher un message indiquant qu'un conflit s'est produit.

Le fichier `index.html` contient alors des marqueurs similaires à :

```
<<<<<<< HEAD
<title>Mon site officiel</title>
=====
<title>Bienvenue sur mon site</title>
>>>>>>> modif-texte
```

Ces marqueurs indiquent les deux versions en conflit.

```
<<<<<<< HEAD
            version actuelle présente sur main
=====
            version provenant de modif-texte
>>>>>>> modif-texte
```

Résoudre le conflit

Le développeur doit décider quelle version conserver.

Par exemple, on peut choisir :

```
<title>Bienvenue sur mon site officiel</title>
```

Il faut supprimer les marqueurs :

```
<<<<<<<
=====
>>>>>>>
```

Le fichier doit ensuite contenir une version valide.

Étape 1 — Indiquer que le conflit est résolu

```
git add index.html
```

Vérifier :

```
git status
```

Étape 2 — Finaliser la fusion

```
git commit -m "Résolution du conflit sur le titre"
```

La fusion est maintenant terminée.

Observer l'historique

Utiliser :

```
git log --oneline --graph
```

Pour obtenir un affichage plus complet :

```
git log --oneline --graph --all
```

Vous pourrez observer les différentes branches et leur fusion.

Exemple :

```
* a123456 Résolution du conflit sur le titre
| \
| * b234567 Modification du titre
* | c345678 Modification du titre sur main
```

```
|/  
* d456789 Ajout de la page d'accueil
```

- Quelle est la différence entre une branche et un commit ?
- Pourquoi Git a-t-il détecté un conflit ?
- Que signifient les marqueurs <<<<<<, ===== et >>>>>> ?
- Pourquoi est-il nécessaire de modifier le fichier avant de faire `git add` ?
- À quoi sert `git merge` ?
- Quelle est la différence entre `git switch` et `git merge` ?

TP 3 : activité de synthèse — compte rendu à rendre

Le TP3 reprend les notions étudiées dans les deux premiers TP. Vous devrez réaliser le projet, conserver un historique Git cohérent et fournir un compte rendu expliquant les principales opérations effectuées.

Objectif

Mettre en œuvre les différentes notions dans un petit projet Web.

Nom du Dépôt : `mon-site-web-simple`

Structure des Fichiers :

```
mon-site-web-simple/  
├─ index.html  
├─ style.css  
├─ script.js  
└─ README.md
```

Contenu de chaque fichier :

1. `index.html` (La page principale du site)

```
<!DOCTYPE html>  
<html lang="fr">  
<head>  
  <meta charset="UTF-8">  
  <meta name="viewport" content="width=device-width, initial-  
scale=1.0">  
  <title>Mon Site Web Simple</title>  
  <link rel="stylesheet" href="style.css">  
</head>  
<body>  
  <header>  
    <h1>Bienvenue sur mon site Web</h1>  
  </header>
```

```
<main>
  <p>Ceci est un exemple de page pour mon site web. Je suis
étudiant en BTS CIEL – IR</p>
  <button id="monBouton">Valider</button>
</main>
<footer>
  <p>&copy; 2026 Mon Site Web</p>
</footer>
<script src="script.js"></script>
</body>
</html>
```

2. **style.css** (Les styles CSS pour le site)

```
body {
  font-family: Arial, sans-serif;
  line-height: 1.6;
  margin: 0;
  padding: 0;
  background: #f4f4f4;
  color: #333;
}

header {
  background: #333;
  color: #fff;
  padding: 1rem 0;
  text-align: center;
}

main {
  padding: 20px;
  text-align: center;
}

button {
  background-color: #007bff;
  color: white;
  padding: 10px 15px;
  border: none;
  cursor: pointer;
  border-radius: 5px;
  margin-top: 15px;
}

footer {
  text-align: center;
  padding: 1rem 0;
  background: #333;
  color: #fff;
  position: fixed;
  width: 100%;
}
```

```
    bottom: 0;
}
```

3. `script.js` (Script JavaScript)

```
document.addEventListener('DOMContentLoaded', () => {
    const monBouton = document.getElementById('monBouton');

    if (monBouton) {
        monBouton.addEventListener('click', () => {
            alert('Bouton cliqué ! Votre requête est enregistrée.');
```

4. `README.md` (Description du projet, affichée sur la page principale du dépôt GitHub)

Mon site web Simple

Ce dépôt contient un exemple simple de site web personnel avec des fichiers HTML, CSS et JavaScript basiques.

Contenu

- ``index.html``: La structure principale de la page web.
- ``style.css``: Les styles visuels pour le site.
- ``script.js``: Un petit script interactif (alerte au clic sur un bouton).
- ``README.md``: Ce fichier, décrivant le projet.

Comment l'utiliser

1. Clonez ce dépôt sur votre machine locale.

```
`git clone https://github.com/votre-pseudo/mon-site-web-simple.git`
```

2. Ouvrir le fichier ``index.html`` dans votre navigateur web.

****Objectif pédagogique****

Ce projet sert d'exemple pour découvrir Git et GitHub, notamment les opérations de clonage, de commit, de push, de pull et de gestion des branches.

Se placer dans le dossier :

```
cd mon-site-web-simple
```

Initialiser Git :

```
git init
```

Vérifier l'état du dépôt :

```
git status
```

Ajouter les fichiers :

```
git add .
```

Créer le premier commit :

```
git commit -m "Création du site Web"
```

Créer le dépôt GitHub

Créer sur GitHub un dépôt appelé : `mon-site-web-simple`

Le dépôt doit rester vide lors de sa création.

N'ajouter pas automatiquement :

```
README ;  
.gitignore ;  
licence.
```

Associer le dépôt local au dépôt GitHub

Copier l'URL fournie par GitHub.

Exemple :

```
git remote add origin https://github.com/votre-pseudo/mon-site-web-
```



```
simple.git
```

Vérifier :

```
git remote -v
```

Publier le projet

Envoyer la branche main vers GitHub :

```
git push -u origin main
```

Actualiser la page GitHub.

Les quatre fichiers doivent apparaître dans le dépôt.

Le contenu du **README.md** est également affiché sur la page principale du dépôt.

Modifier le projet avec Visual Studio Code

Après avoir publié le projet, nous allons effectuer une modification depuis VS Code.

Ouvrir le dossier du projet dans Visual Studio Code.

Modifier par exemple le texte :

```
<h1>Bienvenue sur mon site Web</h1>
```

en :

```
<h1>Bienvenue sur mon premier projet GitHub</h1>
```

Enregistrer le fichier avec : **Ctrl + S** ou **File > Save**

Observer les modifications avec VS Code

Dans Visual Studio Code, ouvrir la vue **Source Control** (trois cercles reliés) dans la barre d'activités à gauche de VS Code. Vous pouvez aussi utiliser le raccourci **Ctrl + Shift + G**.

Le fichier **index.html** apparaît dans la liste des modifications.

Cliquer dessus pour afficher le diff.

Le diff permet de comparer :

- la version enregistrée dans le dernier commit ;
- la version actuellement modifiée.

Ajouter la modification à la zone de staging

Dans la vue Source Control, cliquer sur + à côté du fichier.

Cette opération correspond à :

```
git add index.html
```

Le fichier passe de la liste des modifications à la zone des modifications préparées (Staged Changes).

Créer le commit

Saisir un message explicite, par exemple :

```
Modification du titre de la page
```

Puis créer le commit : Cliquer sur le bouton "Commit" (l'icône de coche) situé juste au-dessus de la zone de message.

Cela correspond à :

```
git commit -m "Modification du titre de la page"
```

Envoyer le commit vers GitHub

Utiliser le bouton Push de VS Code.

Ou utiliser le terminal :

```
git push
```

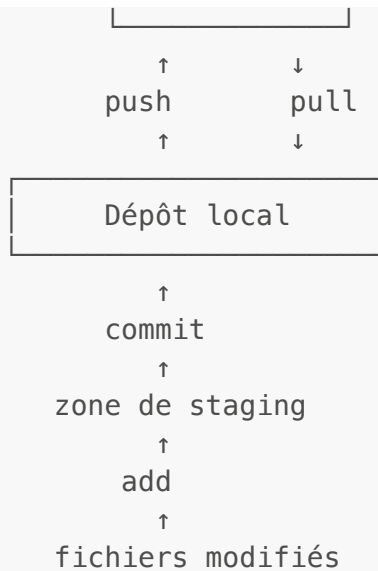
Actualiser ensuite la page du dépôt GitHub.

La modification doit apparaître dans le projet distant.

Résumé

GITHUB

Dépôt
distant



Pour obtenir une première copie locale d'un projet existant :



Conclusion

GIT.	
git status	État du dépôt
git add fichier	Ajouter au staging
git add .	Ajouter les fichiers
git commit -m "message"	Créer un commit
git log --oneline	Voir l'historique
git branch	Voir les branches
git switch branche	Changer de branche
git switch -c branche	Créer une branche
git merge branche	Fusionner une branche
git clone URL	Cloner un dépôt
git remote -v	Voir les remotes
git push	Envoyer vers GitHub
git pull	Récupérer de GitHub

Git permet de gérer l'évolution d'un projet grâce à son historique.

GitHub permet notamment de stocker des dépôts Git à distance et de collaborer.

Les opérations essentielles sont :

```
git add  
↓  
git commit  
↓  
git push
```

pour préparer, enregistrer puis envoyer son travail.

Pour récupérer le travail présent sur le dépôt distant :

```
git pull
```

Pour obtenir une première copie d'un dépôt distant :

```
git clone
```

Enfin, les branches permettent de travailler sur différentes évolutions d'un projet et git merge permet de les réunir.

En cas de conflit, Git ne choisit pas automatiquement quelle version conserver : le développeur doit analyser les modifications, résoudre le conflit puis valider la résolution.

-
- La documentation officielle [Git](#) et [GitHub](#).
-